

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files Google_Stock_Price.csv
Google_Stock_Price.csv(text/csv) - 117583 bytes, last modified: 06/05/2026 - 100% done
Saving Google_Stock_Price.csv to Google_Stock_Price.csv

```
#2. Load Dataset
df = pd.read_csv("Google_Stock_Price.csv", thousands=',')

# Take Open column
data = df['Open'].values.reshape(-1, 1)
```

```
scaler = MinMaxScaler(feature_range=(0,1))
# Clean non-numeric values like 'GOOGL' / NaN from Open column, then scale
data = pd.to_numeric(df['Open'], errors='coerce').dropna().values.reshape(-1, 1)
data_scaled = scaler.fit_transform(data)
```

```
train_size = int(len(data_scaled) * 0.8)

train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]
```

```
def create_dataset(dataset):
    X = []
    y = []

    for i in range(60, len(dataset)):
        X.append(dataset[i-60:i, 0])
        y.append(dataset[i, 0])

    return np.array(X), np.array(y)

X_train, y_train = create_dataset(train_data)
X_test, y_test = create_dataset(test_data)
```

```
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
```

```
model = Sequential()

model.add(SimpleRNN(50, return_sequences=True, input_shape=(60,1)))
model.add(SimpleRNN(50))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to `RNN` layers. It has no effect anymore.
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 60, 50)	2,600
simple_rnn_1 (SimpleRNN)	(None, 50)	5,050
dense (Dense)	(None, 1)	51

Non-trainable params: 0 (0.00 B)

```
model.fit(X_train, y_train, epochs=20, batch_size=32)
```

```

Epoch 1/20
30/30 ————— 4s 24ms/step - loss: 0.0118
Epoch 2/20
30/30 ————— 1s 24ms/step - loss: 0.0014
Epoch 3/20
30/30 ————— 1s 23ms/step - loss: 8.9876e-04
Epoch 4/20
30/30 ————— 1s 24ms/step - loss: 0.0011
Epoch 5/20
30/30 ————— 1s 24ms/step - loss: 9.0152e-04
Epoch 6/20
30/30 ————— 1s 25ms/step - loss: 6.9520e-04
Epoch 7/20
30/30 ————— 1s 25ms/step - loss: 6.2168e-04
Epoch 8/20
30/30 ————— 1s 23ms/step - loss: 6.4182e-04
Epoch 9/20
30/30 ————— 1s 25ms/step - loss: 7.4310e-04
Epoch 10/20
30/30 ————— 2s 37ms/step - loss: 5.3396e-04
Epoch 11/20
30/30 ————— 1s 43ms/step - loss: 5.1124e-04
Epoch 12/20
30/30 ————— 1s 38ms/step - loss: 4.7783e-04
Epoch 13/20
30/30 ————— 1s 25ms/step - loss: 5.3478e-04
Epoch 14/20
30/30 ————— 1s 25ms/step - loss: 4.6624e-04
Epoch 15/20
30/30 ————— 1s 25ms/step - loss: 5.0529e-04
Epoch 16/20
30/30 ————— 1s 25ms/step - loss: 3.8383e-04
Epoch 17/20
30/30 ————— 1s 24ms/step - loss: 3.5978e-04
Epoch 18/20
30/30 ————— 1s 25ms/step - loss: 3.4781e-04
Epoch 19/20
30/30 ————— 1s 25ms/step - loss: 4.0454e-04
Epoch 20/20
30/30 ————— 1s 25ms/step - loss: 7.3090e-04
<keras.src.callbacks.history.History at 0x7dd15a387a40>

```

```

predicted = model.predict(X_test)

predicted = scaler.inverse_transform(predicted)
real = scaler.inverse_transform(y_test.reshape(-1,1))

```

```

6/6 ————— 0s 11ms/step

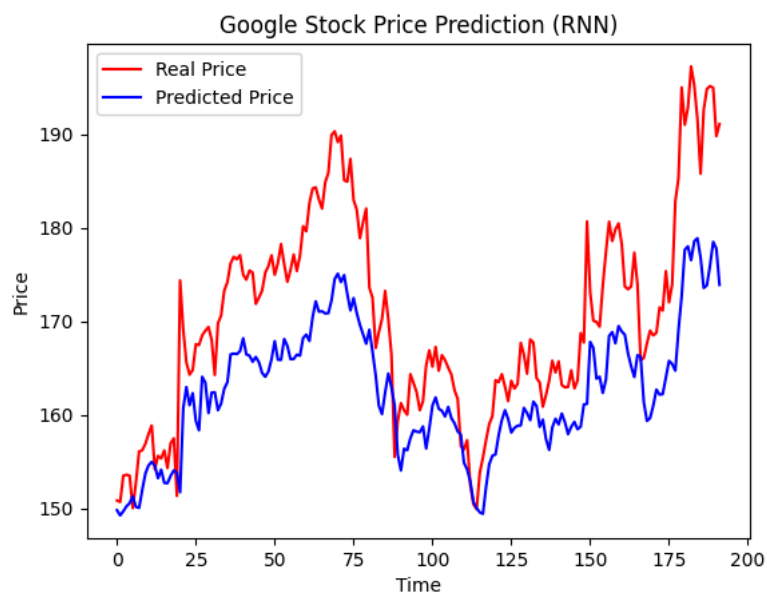
```

```

plt.plot(real, color='red', label='Real Price')
plt.plot(predicted, color='blue', label='Predicted Price')

plt.title("Google Stock Price Prediction (RNN)")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()

```



Start coding or [generate](#) with AI.